

UNIT – I

Overview and History of NoSQL Databases: -

- NoSQL is a type of database management system (DBMS) that is designed to handle and store large volumes of unstructured and semi-structured data.
- Unlike traditional relational databases that use tables with pre-defined schemas to store data, NoSQL databases use flexible data models that can adapt to changes in data structures and are capable of scaling horizontally to handle growing amounts of data.
- The term NoSQL originally referred to “non-SQL” or “non-relational” databases, but the term has since evolved to mean “not only SQL,” as NoSQL databases have expanded to include a wide range of different database architectures and data models.
- NoSQL Database is used to refer a non-SQL or non-relational database.
- It provides a mechanism for storage and retrieval of data other than tabular relations model used in relational databases.
- NoSQL database doesn't use tables for storing data. It is generally used to store big data and real-time web applications.

History behind the creation of NoSQL Databases: -

- In the early 1970, Flat File Systems are used. Data were stored in flat files and the biggest problems with flat files are each company implement their own flat files and there are no standards. It is very difficult to store data in the files, retrieve data from files because there is no standard way to store data.
- Then the relational database was created by E.F. Codd and these databases answered the question of having no standard way to store data. But later relational database also get a problem that it could not handle big data, due to this problem there was a need of database which can handle every types of problems then NoSQL database was developed.

What Is a Database?

- A database management system is a software to store, organize, manage, and retrieve data. Think of it like a group of massive spreadsheets that organize information. There's more than one type of database management system, and each is housed on servers, whether in a data center or virtually, on cloud infrastructure (cloud database).

NoSQL: -

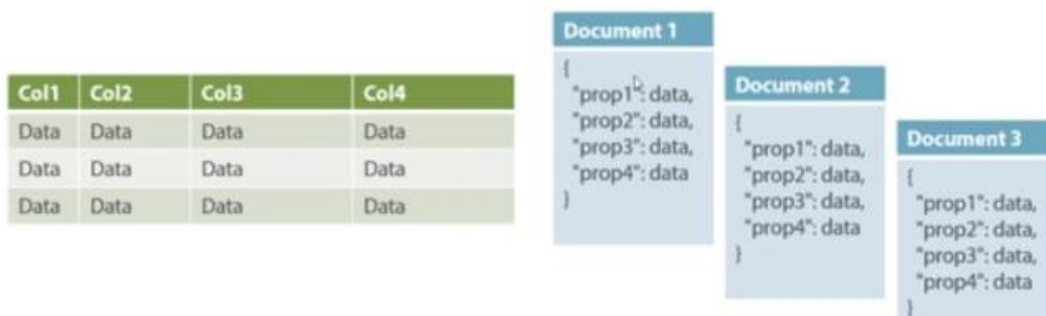
- It provides a mechanism for storage and retrieval of data other than tabular relations model used in relational databases. NoSQL database doesn't use tables for storing data. It is generally used to store big data and real-time web applications.
- NoSQL databases (aka "not only SQL") are non-tabular databases and store data differently than relational tables. NoSQL databases come in a variety of types based on their data model. The main types are document, key-value, wide-column, and graph. They provide flexible schemas and scale easily with large amounts of data and high user loads.

Types of NoSQL databases: -

four major types of NoSQL databases emerged: document databases, key-value databases, wide-column stores, and graph databases.

1. Document Databases

- Document Databases use key-value pairs to store and retrieve data from the documents. A document is stored in the form of XML and JSON. A typical example of the document database is shown below:



The above figure shows that the Document database contains data in JSON (or XML) format and can contain varying schema. The Documents can be nested and indexed for faster querying.

Document databases allow developers to restructure their Documents based on their application requirements which may change over time. In contrast, in the RDBMS world, database administrators are required to restructure the database schemas.

Examples of Document databases are – MongoDB, OrientDB, Apache CouchDB, IBM Cloudant, CrateDB, BaseX, and many more.

It is a type of non-relational database that is designed to store and query data as JSON-like documents which makes it easier for developer to store and query data in a database.

- It works well with use cases such as catalogs, user profiles etc.

- In document store database the data which is collection of key-value pairs is compressed as a document store.
- The flexible, semi-structured and hierarchical nature of documents and document databases allows them to evolve with applications need.
- Example: Book document

```
{
    "id" : "98765432",
    "type" : "book",
    "ISBN": 987-6-543-21012-3,
    "Author":
    {
        "Lname": "Roe",
        "MI": "T",
        "Fname": "Richard"
    },
    "Title": "Understanding document databases"
}
```

2. Key-Value Stores

- Key-value Stores are the simplest type of NoSQL Database. It uses keys and values to store the data. The attribute name is stored in 'key', whereas the values corresponding to that key will be held in 'value'.
- In Key-value store databases, the key can only be string, whereas the value can store string, JSON, XML, Blob, etc. Due to its behavior, it is capable of handling massive data and loads.
- The use case of key-value stores mainly stores user preferences, user profiles, shopping carts, etc.
- DynamoDB, Riak, Redis are a few famous examples of Key-value store NoSQL databases.

Key-value databases are the simplest of the NoSQL databases: The basic data structure is a dictionary or map. You can store a value, such as an integer, string, a JSON structure, or an array, along with a key used to reference that value.

- For example, a simple key-value database might have a value such as "Douglas Adams". This value is then assigned an ID, such as cust1237.
- Using a JSON structure adds complexity to the database. For example, the database could store a full mailing address in addition to a person's name. In the previous example,

key cust1237 could point to the following information:

```
{ name: "Douglas Adams",
  street: "782 Southwest St.",
  city: "Austin",
  state: "TX"
}
```

3. Column-Oriented Databases

- Column-oriented databases store the data in a set of columns known as column families. That means that whenever a user wants to run queries for a smaller number of columns, they can read those columns directly without consuming memories corresponding to all data.
- The working of the Column-oriented database is based on the concept of the BigTable paper by Google. Below schematics shows how values are stored on Column-oriented databases:

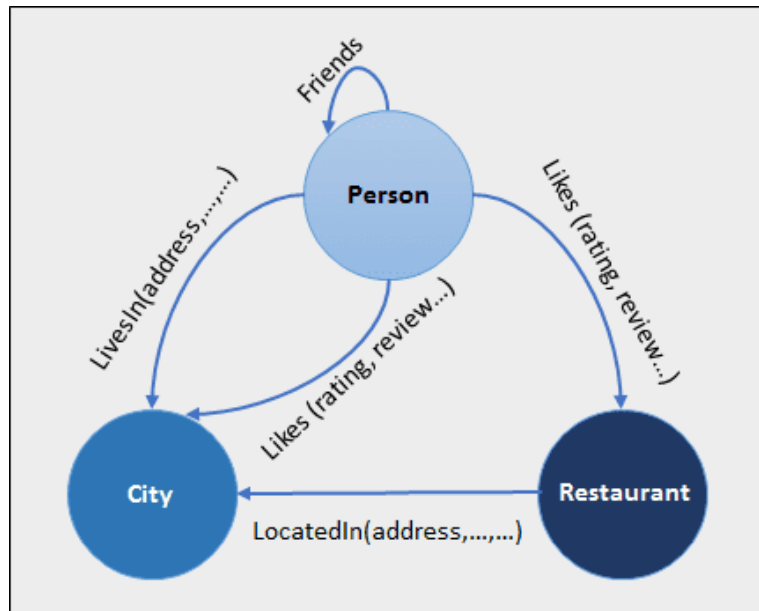
ColumnFamily			
Row Key	Column Name		
	Key	Key	Key
	Value	Value	Value
	Column Name		
	Key	Key	Key
	Value	Value	Value

- HBase, Cassandra, HBase, Hypertable are NoSQL query examples of column-based databases.

4. Graph Databases

- Graph databases form and store the relationship of the data. Each element/data is stored in a node, and that node is linked to another data/element.
- A typical example for Graph database use cases is Facebook. It holds the relationship between each user and their further connections.

Graph databases help search the connections between data elements and link one part to various parts directly or indirectly.



1. The Graph database can be used in social media, fraud detection, and knowledge graphs. Examples of Graph Databases are – Neo4J, Infinite Graph, OrientDB, FlockDB, etc.

The Value of Relational Databases

1. Getting at Persistent Data

Two areas of memory:

- Fast, small, volatile main memory
- Larger, slower, non volatile backing store.
- Since main memory is volatile to keep data around, we write it to a backing store, commonly seen a disk which can be persistent memory.

The backing store can be:

- File system
- Database

- The database allows more flexibility than a file system in storing large amounts of data in a way that allows an application program to get information quickly and easily.

2. Concurrency

- Enterprise applications tend to have many people using same data at once, possibly modifying that data. We have to worry about coordinating interactions between them to avoid things like double booking of hotel rooms.
- Since enterprise applications can have lots of users and other systems all working concurrently, there's a lot of room for bad things to happen. Relational databases help to handle this by controlling all access to their data through transactions.

3. Integration

- Enterprise requires multiple applications, written by different teams, to collaborate in order to get things done. Applications often need to use the same data and updates made through one application have to be visible to others.
- A common way to do this is shared database integration where multiple applications store their data in a single database.
- Using a single database allows all the applications to use each other's data easily, while the database's concurrency control handles multiple applications in the same way as it handles multiple users in a single application.

Impedance Mismatch

- Impedance mismatch is a term used in computer science to describe the problem that arises when two systems or components that are supposed to work together have different data models, structures, or interfaces that make communication difficult or inefficient.
- In the context of databases, impedance mismatch refers to the discrepancy between the object-oriented programming (OOP) model used in application code and the relational model used in database management systems (DBMS). While OOP models are designed to represent data as objects with properties and methods, relational models represent data as tables with columns and rows.
- Impedance mismatch is the term used to refer to the problems that occurs due to differences between the database model and the programming language model. The practical relational model has 3 components these are:
 1. Attributes and their data types
 2. Tuples
 3. Tables

Problems: Following problems may occur due to the impedance mismatch:

1. The first problem that may occur is that is data type mismatch means the programming language attribute data type may differ from the attribute data type in the data model.
2. The second problem that may occur is because the results of most queries are sets or multisets of tuples and each tuple is formed of a sequence of attribute values. In the program, it is necessary to access the individual data values within individual tuples for printing or processing.

Application and Integration Databases

- In relational databases, the database acts as an integration database—where multiple applications developed by separate teams storing their data in a common database. This improves communication because all the applications are operating on a consistent set of persistent data.

There are downsides to shared database integration.

- A structure that's designed to integrate many applications is more complex than any single application needs.
- If an application wants to make changes to its data storage, it needs to coordinate with all the other applications using the database.
- Different applications have different structural and performance needs, so an index required by one application may cause a problematic hit on inserts for another.
- A different approach is to treat your database as an application database—which is only accessed by a single application codebase that's looked after by a single team.

Advantages:

- With an application database, only the team using the application needs to know about the database structure, which makes it much easier to maintain and evolve the schema.
- Since the application team controls both the database and the application code, the responsibility for database integrity can be put in the application code.

The Emergence of NoSQL

- For NoSQL there is no generally accepted definition, nor an authority to provide one, so all we can do is discuss some common characteristics of the databases that tend to be called "NoSQL."
- The name NoSQL comes from the fact that the NoSQL databases doesn't use SQL as a query language. Instead, the database is manipulated through shell scripts that can be combined into the usual UNIX pipelines.
- They are generally open-source projects.

- Most NoSQL databases are driven by the need to run on clusters. Relational databases use ACID transactions to handle consistency across the whole database. This inherently clashes with a cluster environment, so NoSQL databases offer a range of options for consistency and distribution.
- Not all NoSQL databases are strongly oriented towards running on clusters. Graph databases are one style of NoSQL databases that uses a distribution model similar to relational databases but offers a different data model that makes it better at handling data with complex relationships.
- NoSQL databases operate without a schema, allowing you to freely add fields to database records without having to define any changes in structure first. This is particularly useful when dealing with non uniform data and custom fields which forced relational databases to use names like customField6 or custom field tables that are awkward to process and understand.
- When you first hear “NoSQL,” an immediate question is what does it stand for—a “no” to SQL? Most people who talk about NoSQL say that it really means “Not Only SQL,” but this interpretation has a couple of problems. Most people write “NoSQL” whereas “Not Only SQL” would be written “NOSQL.”
- To resolve these problems, don’t worry about what the term stands for, but rather about what it means. Thus, when “NoSQL” is applied to a database, it refers to an ill-defined set of mostly open-source databases, mostly developed in the early 21st century, and mostly not using SQL.
- It’s better to think of NoSQL as a movement rather than a technology. We don’t think that relational databases are going away—they are still going to be the most common form of database in use. Their familiarity, stability, feature set, and available support are compelling arguments for most projects.
- The change is that now we see relational databases as one option for data storage. This point of view is often referred to as polyglot persistence—using different data stores in different circumstances.
- We need to understand the nature of the data we’re storing and how we want to manipulate it. The result is that most organizations will have a mix of data storage technologies for different circumstances. In order to make this polyglot world work, our view is that organizations also need to shift from integration databases to application databases.
- In our account of the history of NoSQL development, we’ve concentrated on big data running on clusters. The big data concerns have created an opportunity for people to think freshly about their data storage needs, and some development teams see that using a NoSQL database can help their productivity by simplifying their database access even if they have no need to scale beyond a single machine.

Two primary reasons for considering NoSQL:

- 1) To handle data access with sizes and performance that demand a cluster
- 2) To improve the productivity of application development by using a more convenient data interaction style.

- A NoSQL is a database that provides a mechanism for storage and retrieval of data, they are used in real-time web applications and big data and their use are increasing over time.
- Many NoSQL stores compromise consistency in favor of availability, speed and partition tolerance.

Advantages of NoSQL:

1. High Scalability

NoSQL databases use sharding for horizontal scaling. It can handle huge amount of data because of scalability, as the data grows NoSQL scale itself to handle that data in efficient manner.

2. High Availability

Auto replication feature in NoSQL databases makes it highly available.

Disadvantages of NoSQL:

1. Narrow Focus: It is mainly designed for storage, but it provides very little functionality.
2. Open Source: NoSQL is open-source database that is two database systems are likely to be unequal.
3. Management Challenge: Big data management in NoSQL is much more complex than a relational database.
4. GUI is not available: GUI mode tools to access the database is not flexibly available in the market.
5. Backup: it is a great weak point for some NoSQL databases like MongoDB.
6. Large Document size: Data in JSON format increases the document size.

When should NoSQL be used

- When huge amount of data need to be stored and retrieved.
- The relationship between data you store is not that important.
- The data changing over time and is not structured.
- Support of constraint and joins is not required at database level.
- The data is growing continuously and you need to scale the database regular to handle the data.

Key Points

- Relational databases have been a successful technology for twenty years, providing persistence, concurrency control, and an integration mechanism.
- Application developers have been frustrated with the impedance mismatch between the relational model and the in-memory data structures.
- There is a movement away from using integration databases towards encapsulating databases within applications and integrating through services.
- The vital factor for a change in data storage was the need to support large volumes of data by running on clusters. Relational databases are not designed to run efficiently on clusters.

The common characteristics of NoSQL databases

1. Not using the relational model
2. Running well on clusters
3. Open-source
4. Built for the 21st century web estates
5. Schema less
6. The most important result of the rise of NoSQL is Polyglot Persistence.

SQL Databases used to be the primary way in which almost all the organizations stored their data and built applications on top of it. However, the inherent weaknesses in the way in which these databases work and the huge explosion of unstructured and semi-structured data due to the rise of the Internet led to an acute need for a new type of database technology. This is where NoSQL or 'Not Only SQL' databases emerged and filled that vacuum. NoSQL, or Not only SQL, refers to non-relational databases which can efficiently handle large amounts of data and overcome the drawbacks of RDBMS. Any non-tabular form of database storage structure is called a NoSQL database.

Why NoSQL?

To understand why we need NoSQL databases let's start with the disadvantages of the SQL databases. SQL had two main disadvantages:

- The necessity of joins: To get information about a particular record, you must join multiple tables to get relevant data. As the number of tables increases, join becomes a computationally expensive operation, and, therefore, it can slow down your systems.
- Lack of a flexible schema: SQL databases require you to mention all field values explicitly, even if you do not need them for a particular record.

NoSQL databases on the other hand store documents in a denormalized structure and also have a flexible schema. For example, in the following document, all the information pertaining to a specific Order_ID is at a single place. You don't need to perform joins to get the relevant data from multiple tables.

```
{
  "_id" : ObjectId("5eec2adaee4bfd8f653b24dc"),
  "Order_ID" : "TEC-103",
  "Name" : "Plantronics",
  "Sales" : 2400,
  "Profit" : 760
}
```

SQL vs NoSQL Databases

Factors	SQL	NoSQL
Schema	SQL databases store information in the form of tables with a fixed schema.	NoSQL databases allow a dynamic schema.
Scaling	SQL databases are generally scaled vertically by increasing the CPU and RAM to increase the power of the existing database.	NoSQL databases are more suitable for horizontal scaling, where you keep adding more servers to improve performance.
Structure	SQL databases are designed to handle advanced querying requirements.	NoSQL databases are utilized to handle complex data, and they can be scaled as per requirement.

Depending on your use case you will need to take a call if you'll go with SQL or NoSQL databases for your storage needs.

CAP Theorem

The CAP theorem, which one can use to select the right database for a given application. The choice of database is not only between the SQL and NoSQL as mentioned above but also between the various types of NoSQL databases.

CAP stands for

- **Consistency:** The data available is consistent across all the nodes or servers.
- **Availability:** The data is available for querying irrespective of whether or not it is the latest version of the data.
- **Partition tolerance:** The system continues to operate despite network failures (ie; dropped partitions, slow network connections, or unavailable network connections between nodes).

The CAP theorem states that it is impossible to achieve both consistency and availability in a partition tolerant distributed system (i.e., a system which continues to work in cases of temporary communication breakdowns).

According to the CAP theorem, SQL databases focus on consistency and availability. They fall on the Consistent and Available (CA) spectrum of databases. On the other hand, since NoSQL databases are distributed across multiple servers as partitions, partition tolerance is something they cannot ignore. So the majority of the NoSQL databases fall under the Consistent and Partition tolerant (CP) or Available and Partition Tolerant (AP) spectrum of databases.

A direct application of this theorem is used when deciding which database management system better suits your needs. If you choose consistency over availability, you choose a certain database management system and vice versa.

BASE Model

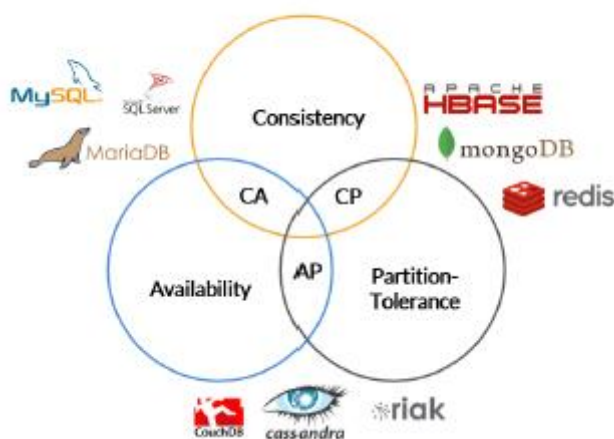
Earlier in the program, you learned the ACID model when dealing with SQL databases in the first course. ACID is an acronym that stands for Atomicity, Consistency, Isolation, and Durability. The ACID model focuses more on consistency over availability. SQL databases are based on the ACID model. On the other hand NoSQL databases follow a BASE consistency model.

BASE stands for:

- **Basically available:** The system is guaranteed to be available in the event of failure.
- **Soft state:** The state of the data could change without application interactions due to eventual consistency.
- **Eventually consistent:** The system will be eventually consistent after the application input. The data will be replicated to different nodes and will eventually reach a consistent state.

Using these two concepts you should be able to pick the right database management system that would best suit your storage needs. Here is an illustration that would help you map the concepts discussed above and the DBMS that best suits that particular use case.

DATABASES AND THEIR USE CASES



Types of NoSQL Databases and Use Cases

NoSQL databases are of the following four basic types:

- **Key-value stores:** In this type of NoSQL database, data is stored as keys and their values. Each data element is associated with a pointer and a unique identifier. Arbitrary strings are used as keys, and the value could be a document or an image. Key-value data stores have large hash tables, which contain the keys and the values. Hash tables are data structures that map the keys to their corresponding values. Some popular key-value NoSQL data stores include Cassandra and Redis.
- **Document-based stores:** Each record and all associated data are stored within a document in this type of NoSQL database. The documents are made up of tagged elements. Some examples of document-based data stores include CouchDB and MongoDB.
- **Column-based stores:** In this type of NoSQL database, data is stored by column rather than by row. Each storage block contains data from only one column. HBase and Hypertable are popular examples of column-based data stores.
- **Graph-based stores:** Network databases use edges and nodes to represent and store data. A popular example is Neo4J.

Here are some use cases where NoSQL databases outperform traditional databases:

- Storage of real-time big-data
- Internet of things

- Customer 360° View
- Handling semi-structured data